CONFINAMENTO: APPARMOR, SELINUX E SANDBOXING

*Às 3h47 o invasor executou código dentro do `nginx`. O roteiro é sempre o mesmo: ler `/etc/passwd`, achar chave privada em `/home`, abrir conexão de saída. **As permissões Unix teriam deixado passar** — o processo era o dono. Barrou um perfil que listava, arquivo por arquivo, o que aquele programa podia tocar.*

088
GUIAS LINUX

DOIS PORTÕES EM SÉRIE — E BASTA UM NEGAR

PROCESSO
o pedido ao kernel

```
open("/home/x/.ssh/id_rsa")
```

→

PORTÃO DAC
dono, grupo, outros · ACL · SUID.
DISCRICIONÁRIO:
o dono decide, e `chmod 777` abre.
"quem você é?"
`nega → EACCES`

→

PORTÃO MAC
perfil AppArmor ou tipo SELinux.
OBRIGATÓRIO:
política do administrador; nem `root` afrouxa em execução.
"o que você é?"
`nega → EACCES + registro`

→

RECURSO
arquivo, socket, capability

Os dois portões são verificados em série e de forma independente: a chamada só chega ao recurso se **ambos** permitirem. Afrouxar o DAC não abre o MAC, e ser `root` não abre nenhum dos dois quando há política. O invasor que ganhou execução no processo herda a identidade dele — e portanto o DAC —, mas **não escapa do perfil: o perfil descreve o programa, não o usuário**. E a negação do MAC **deixa registro** — é bloqueio e detecção ao mesmo tempo.

A MESMA IDEIA, DUAS ESCOLHAS OPOSTAS DE PROJETO

ASPECTO	APPARMOR	SELINUX
IDENTIFICA POR	**caminho** do arquivo	**rótulo** no atributo estendido
PADRÃO EM	Debian, Ubuntu, **Kali**, SUSE	RHEL, Fedora, CentOS, Android
POLÍTICA EM	texto legível, parecido com shell	linguagem própria, compilada
APRENDER LEVA	horas	semanas
PONTO FORTE	confinar um programa, rápido	política do sistema inteiro
PONTO FRACO	cópia ou link com outro caminho escapa	diagnóstico difícil no início

NA PRÁTICA — E COMO SE LÊ UMA NEGAÇÃO

kali@kali: ~

```
# AppArmor — no Kali pode estar instalado e NÃO ativo
$ sudo aa-status
apparmor module is not loaded  ← perfil em disco
não é política em vigor
$ sudo aa-genprof /usr/local/bin/servico # esqueleto
$ sudo aa-complain ...; sudo aa-logprof # aprende
$ sudo aa-enforce ...   # só então, aplica
$ sudo dmesg | grep 'apparmor="DENIED"'
# SELinux (RHEL) — o mesmo, porém por rótulo
$ getenforce; ls -Z /var/www/html; id -Z
$ sudo ausearch -m AVC -ts recent   # as negações
$ sudo restorecon -Rv /var/www/html # rótulo certo
$ firejail --net=none --private ./binario-desconhecido
```

A LINHA QUE DENUNCIOU O INCIDENTE DAS 3H47

```
apparmor="DENIED" operation="open" profile="/usr/sbin/nginx" name="/home/deploy/.ssh/id_rsa" requested_mask="r" pid=4711 comm="nginx"
```

`profile`	**quem** foi barrado — o **programa**, não o usuário
`name`	**o que** ele tentou tocar
`requested_mask`	com **qual** permissão (`r`, `w`, `x`)

Um `nginx` não tem motivo nenhum para procurar chave privada em `/home`. Perfil bem construído gera **zero** negações em operação normal — o que torna *qualquer* negação um evento a investigar. Perfil ruidoso treina o operador a ignorar o log.

Sandbox não é ambiente de análise de *malware*. `firejail`, `bubblewrap` e contêineres **compartilham o kernel** com o hospedeiro, e uma falha de kernel escapa de todos eles. Sandbox serve para reduzir o dano de um programa que você *espera* ser bem-intencionado; para executar código que você **sabe** ser hostil, o isolamento correto é **VM descartável, em rede isolada, com snapshot e sem pasta compartilhada**. Confundir os dois transforma uma análise de amostra num incidente na sua estação.

RESUMO RÁPIDO

- ✓ **DAC: o dono decide** — e o processo comprometido herda toda a autoridade do usuário. **MAC: o administrador decide** o que cada *programa* pode fazer, e nem `root` afrouxa isso em execução.
- ✓ **Os dois portões são independentes:** `chmod 777` não abre o MAC. O invasor precisa vencer os dois.
- ✓ **AppArmor identifica por caminho; SELinux, por rótulo.** Caminho é fácil de escrever e contornável por cópia; rótulo viaja com o arquivo, ao custo de uma curva bem maior.
- ✓ **Modo `complain` constrói perfil sem quebrar serviço** — registra tudo, bloqueia nada. O caminho de código que você não exercitou vai quebrar depois, às 3h da manhã.
- ✓ **`Permission denied` com permissões corretas** é o sintoma clássico: `dmesg | grep DENIED` no AppArmor, `ausearch -m AVC` no SELinux.
- ✓ **Sandbox reduz dano de programa falho; VM isola código hostil.** Não são intercambiáveis.

NO KALI

O AppArmor vem instalado no Kali, mas nem sempre ativo — confira antes de supor. Um `aa-status` pode responder `apparmor module is not loaded`; ativar exige `apparmor=1 security=apparmor` em `GRUB_CMDLINE_LINUX_DEFAULT`, `update-grub` e reboot. Sem isso, os perfis em `/etc/apparmor.d/` estão em disco e não valem nada — **a situação mais enganosa possível, porque a configuração parece feita. Confinar a ferramenta ofensiva é tão legítimo quanto confinar o serviço:** boa parte do que roda num Kali processa entrada não confiável da rede — `tcpdump` decodifica pacote de terceiro, o Wireshark tem uma superfície enorme de decodificadores, `sqlmap` e `nikto` consomem resposta de servidor alheio —, fontes historicamente generosas de CVEs. O perfil `usr.sbin.tcpdump` já vem pronto e mostra por que perfil + capability é a defesa completa: a capability dá só o privilégio necessário, o perfil limita o alcance se o processo for explorado. `aa-complain` **é a resposta certa** para "o AppArmor está quebrando meu serviço" — mantém o registro e para de bloquear, permitindo diagnosticar sem desligar a proteção do resto do sistema; `aa-disable` por um perfil problemático é jogar a camada fora por um caso pontual. **RHEL inverte o padrão:** SELinux em *Enforcing* desde a instalação, e quem vem do Debian tropeça uma dezena de vezes no mesmo sintoma — serviço com permissão perfeita e comportamento inexplicável — antes de criar o reflexo: **antes de investigar permissão,** `ausearch -m AVC -ts recent`. Havendo AVC, o problema é rótulo, e `restorecon -Rv` ou um booleano resolve numa linha. `setenforce 0` "só para testar" vale se vier seguido de `setenforce 1`; `SELINUX=disabled` no arquivo, nunca. E `audit2allow` é diagnóstico, não solução: transforma **toda** negação em permissão — inclusive a que era o ataque.